

una función es un bloque de código que realiza una tarea específica cuando una función es llamada (invocada)

Existen tipos de funciones básicas:

Funciones integradas como lo es `print()`

Módulos Pre instalados

Funciones definidas por el usuario

Las funciones lambda.

Las funciones propias se pueden definir con `def`

Una función con parámetro

```
def hi(name):
```

```
    print("Hola, ", name)
```

```
hi("Sergio")
```

Puedes usar argumentos

Para los argumentos posicionales el orden de los parámetros es relevante

Para los argumentos con palabras clave en el cual el orden de los argumentos posicionales y después los de palabras clave.

Puedes emplear la palabra reservada `return` para decirle a una función que devuelva algún valor la instrucción `return` termina la función

Si es el resultado de una función se puede asignar fácilmente a una variable.

Puedes fácilmente usar una lista como argumento de una función

Una función puede invocar otras funciones o incluso a sí misma cuando una función se invoca a sí misma se le conoce como recursividad y la función que se invoca a sí misma y contiene una condición de terminación es llamada una función recursiva.

Se pueden emplear funciones recursivas en Python para crear los árboles más grandes. Sin embargo se debe tener mucho cuidado ya que es muy fácil cometer errores y crear una función la cual nunca termine también se debe considerar que las funciones recursivas consumen mucha memoria y por lo tanto puede ser en ocasiones ineficientes.

Las tuplas son colecciones de datos ordenados e inmutables se pueden pensar en ellos como listas inmutables se definen con paréntesis.

Cada elemento de la tupla puede ser un tipo de dato diferente las tuplas pueden contener otras tuplas o listas (y viceversa).

Las diccionarios son colecciones indexadas de datos mutables y desordenadas si se desea acceder a un elemento del diccionario del diccionario se puede hacer una referencia a su clave colocándola dentro de corchetes o utilizando el método `get()`.

Gras a sintaxis de Python, como modo de analizarlos en nuestra mente la sintaxis de código que nos afecta o vice incluso cuando la sintaxis o expresiones es sintácticamente correcta.



Tu puntaje: 12/12

100%

Congratulations, you've passed the quiz!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 4 Quiz 100%

Volver a presentar la prueba

Revisar prueba

R

Ricardo Daniel
1225100759.rdi@gmail.com

- Iniciar sesión en tu teléfono
- Contraseñas y Autocompletar
- Gestionar tu cuenta de Google
- Personalizar perfil
- La sincronización está activada

- Añadir perfil de Chrome
- Perfil de invitado abierto
- Gestionar perfiles de Chrome



Tu puntaje: 21/22

95%

Congratulations, you've passed the test!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 4 Test 95%

Volver a presentar la prueba

Revisar prueba

R

Ricardo Daniel
1225100759.rdi@gmail.com

- Iniciar sesión en tu teléfono
- Contraseñas y Autocompletar
- Gestionar tu cuenta de Google
- Personalizar perfil
- La sincronización está activada

- Añadir perfil de Chrome
- Perfil de invitado abierto
- Gestionar perfiles de Chrome

Python
INSTITUTE

4.3.1.6 LABORATORIO: Un año bisiesto: escribiendo tus propias funciones

MODULE (32%)

SECTION (58%)

LABORATORIO

Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Proyectar y escribir funciones con parámetros.
- Utilizar la instrucción `return`.
- Probar las funciones.

Escenario

Tu tarea es escribir y probar una función que toma un argumento (un año) y devuelve `True` si el año es un año bisiesto. o `False` si no lo es.

Parte del esqueleto de la función ya está en el editor.

Nota: también hemos preparado un breve código de prueba, que puedes utilizar para probar tu función.

El código utiliza dos listas: una con los datos de prueba y la otra con los resultados esperados. El código te dirá si alguno de tus resultados no es válido.

```
1 def is_year_leap(year):
2     if year % 4 == 0:
3         if year % 100 == 0:
4             if year % 400 == 0:
5                 return True
6             else:
7                 return False
8         else:
9             return True
10    else:
11        return False
12
13 test_data = [1900, 2000, 2016, 1987]
14 test_results = [False, True, True, False]
15 for i in range(len(test_data)):
16     yr = test_data[i]
17     print(yr, "->", end="")
18     result = is_year_leap(yr)
19     if result == test_results[i]:
20         print("OK")
21     else:
22         print("Fallido")
23
```

Console >_

1900 ->OK
2000 ->OK
2016 ->OK
1987 ->OK

Ricardo Daniel
1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

Python
INSTITUTE

4.3.1.7 LABORATORIO: ¿Cuántos días?: escribiendo y utilizando tus propias funciones

MODULE (32%)

SECTION (64%)

LABORATORIO

Tiempo Estimado

15-20 minutos

Nivel de Dificultad

Medio

Requisitos Previos

LABORATORIO 4.1.3.6

Objetivos

Familiarizar al estudiante con:

- Proyectar y escribir funciones parametrizadas.
- Utilizar la instrucción `return`.
- Utilizar las funciones propias del estudiante.

Escenario

Tu tarea es escribir y probar una función que toma dos argumentos (un año y un mes) y devuelve el número de días del mes/año dado (mientras que solo febrero es sensible al valor `year`, tu función debería ser universal).

La parte inicial de la función está lista. Ahora, haz que la función devuelva `None` si los argumentos no tienen sentido.

Por supuesto, puedes (y debes) utilizar la función previamente escrita y probada (LABORATORIO 4.1.3.6). Puede ser muy útil. Te recomendamos que utilices una lista con los meses. Puedes crearla dentro de la función: este truco acortará significativamente el código.

Hemos preparado un código de prueba. Ampliilo para incluir más casos de prueba.

```
2 def is_year_leap(year):
3     if year % 4 != 0:
4         return False
5     elif year % 100 != 0:
6         return True
7     elif year % 400 != 0:
8         return False
9     else:
10        return True
11
12 def days_in_month(year, month):
13     if year < 1 or month < 1 or month > 12:
14         return None
15
16     days = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
17
18     if month == 2 and is_year_leap(year):
19         return 29
20
21     return days[month - 1]
22
23
24 test_years = [1900, 2000, 2016, 1987]
25 test_months = [2, 2, 1, 11]
26 test_results = [29, 29, 31, 30]
27
28 for i in range(len(test_years)):
29     yr = test_years[i]
30     mo = test_months[i]
31     print(yr, mo, "->", end="")
32     result = days_in_month(yr, mo)
33     if result == test_results[i]:
34         print("OK")
35     else:
36         print("Fallido")
```

Console >_

1900 2 ->OK
2000 2 ->OK
2016 1 ->OK
1987 11 ->OK

Ricardo Daniel
1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

Python
INSTITUTE

4.3.1.8 LABORATORIO: Día del año: escribiendo y utilizando tus propias funciones

MODULE (33%)

SECTION (73%)

LABORATORIO

Tiempo Estimado

20-30 minutos

Nivel de Dificultad

Medio

Requisitos Previos

LABORATORIO 4.1.3.6
LABORATORIO 4.1.3.7

Objetivos

Familiarizar al estudiante con:

- Proyectar y escribir funciones con parámetros.
- Utilizar la sentencia `return`.
- Construir un conjunto de funciones de utilidad.
- Utilizar las funciones propias del estudiante.

Escenario

Tu tarea es escribir y probar una función que toma tres argumentos (un año, un mes y un día del mes) y devuelve el día correspondiente del año, o devuelve `None` si cualquiera de los argumentos no es válido.

Debes utilizar las funciones previamente escritas y probadas. Agrega algunos casos de prueba al código. Esta prueba es solo el comienzo.

```
1 def is_year_leap(year):
2     if year % 4 != 0:
3         return False
4     elif year % 100 != 0:
5         return True
6     elif year % 400 != 0:
7         return False
8     else:
9         return True
10
11
12 def days_in_month(year, month):
13     if year < 1 or month < 1 or month > 12:
14         return None
15
16     days = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
17
18     if month == 2 and is_year_leap(year):
19         return 29
20
21     return days[month - 1]
22
23
24 def day_of_year(year, month, day):
25     # Validar el mes y el día usando las funciones previas
26     if year < 1 or month < 1 or month > 12:
27         return None
28
29     total_days = days_in_month(year, month)
30     if total_days is None or day < 1 or day > total_days:
31         return None
32
33     # Sumar los días de los meses anteriores
34     days_sum = 0
35     for m in range(1, month):
```

Console >_

366
366

Ricardo Daniel
1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

Python

INSTITUTE

MODULE (35%)

SECTION (82%)

LABORATORIO

Tiempo Estimado

15-20 minutos

Nivel de Dificultad

Medio

Objetivos

- Familiarizar al estudiante con nociones y algoritmos clásicos.
- Mejorar las habilidades del estudiante para definir y emplear funciones.

Escenario

Un número natural es **primo** si es mayor que 1 y no tiene divisores más que 1 y si mismo.

¿Complicado? De ninguna manera. Por ejemplo, 8 no es un número primo, ya que puedes dividirlo entre 2 y 4 (no podemos usar divisores iguales a 1 y 8, ya que la definición lo prohíbe).

Por otra parte, 7 es un número primo, ya que no podemos encontrar ningún divisor para él.

Tu tarea es escribir una función que verifique si un número es primo o no.

La función:

- Se llama `is_prime`.
- Toma un argumento (el valor a verificar).
- Devuelve `True` si el argumento es un número primo, y `False` de lo contrario.

Sugerencia: intenta dividir el argumento por todos los valores posteriores (comenzando desde 2) y verifica el resto: si es cero, tu número no puede ser un número primo; analiza cuidadosamente cuándo deberías detener el proceso.

1<def is_prime(num):

2< if num <= 1:

3< return False

4< for i in range(2, int(num ** 0.5) + 1):

5< if num % i == 0:

6< return False

7< return True

8<

9<for i in range(1, 20):

10< if is_prime(i + 1):

11< print(i + 1, end=" ")

12<print()

Console >...

2 3 5 7 11 13 17 19

Ricardo Daniel

1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

Python

INSTITUTE

MODULE (35%)

SECTION (91%)

LABORATORIO

Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

- Mejorar las habilidades del estudiante para definir, utilizar y probar funciones.

Escenario

El consumo de combustible de un automóvil se puede expresar de muchas maneras diferentes. Por ejemplo, en Europa, se muestra como la cantidad de combustible consumido por cada 100 kilómetros.

En los EE. UU., se muestra como la cantidad de millas recorridas por un automóvil con un galón de combustible.

Tu tarea es escribir un par de funciones que conviertan l/100km a mpg (millas por galón), y viceversa.

Las funciones:

- Se llaman `liters_100km_to_miles_gallon` y `miles_gallon_to_liters_100km` respectivamente.
- Toman un argumento (el valor correspondiente a sus nombres).

Complementa el código en el editor.

Ejecuta tu código y verifica si tu salida es la misma que la nuestra.

Aquí hay información para ayudarte:

- 1 milla = 1609.344 metros.
- 1 galón = 3.785411784 litros.

1<def liters_100km_to_miles_gallon(liters):

2< # 100 km en millas = 100,000 metros / 1609.344 metros por milla

3< miles = 100 * 1000 / 1609.344

4< # 1 galón en litros = 3.785411784 litros

5< gallons = liters / 3.785411784

6< return miles / gallons

7<

8<def miles_gallon_to_liters_100km(miles):

9< # 1 galón en litros = 3.785411784 litros

10< liters = 3.785411784

11< # 1 milla en 100 km = 100 km / (miles * 1609.344 metros / 100,000)

12< # Equivalente directo: Convertir millas por galón a litros por 100 km

13< # 100 km = 100,000 / 1609.344 millas

14< km_miles = 100 * 1000 / 1609.344

15< gallons = 1

16< return (3.785411784 * 100 * 1000 / 1609.344) / miles

17<

18<print(liters_100km_to_miles_gallon(3.9))

19<print(liters_100km_to_miles_gallon(7.5))

20<print(liters_100km_to_miles_gallon(10.1))

21<print(miles_gallon_to_liters_100km(60.9))

22<print(miles_gallon_to_liters_100km(31.4))

23<print(miles_gallon_to_liters_100km(23.5))

24<print(miles_gallon_to_liters_100km(23.5))

Console >...

60.31143162393162

31.36194444444444

23.52145833333333

3.86073938617467

7.49910297239915

10.009132205673757

Ricardo Daniel

1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

Python

INSTITUTE

MODULE (35%)

SECTION (93%)

PROYECTO

Tiempo Estimado

30-60 minutos

Nivel de Dificultad

Medio/Difícil

Objetivos

- Perfeccionar las habilidades del estudiante al emplear Python para resolver problemas complejos.
- La integración de técnicas de programación en un solo programa consistente de varias partes.

Escenario

Tu tarea es escribir un **simple programa que simule jugar a tic-tac-toe (nombre en inglés) con el usuario**. Para hacerlo más fácil, hemos decidido simplificar el juego. Aquí están nuestras reglas:

- La máquina (por ejemplo, el programa) jugará utilizando las 'X' s.
- El usuario (por ejemplo, tú) jugarás utilizando las 'O' s.
- El primer movimiento es de la máquina: siempre coloca una 'X' en el centro del tablero.
- Todos los cuadros están numerados comenzando con el 1 (observa el ejemplo para que tengas una referencia).
- El usuario ingresa su movimiento introduciendo el número de cuadro elegido. El número debe de ser válido, por ejemplo un valor entero mayor que 0 y menor que 10, y no puede ser un cuadro que ya esté ocupado.
- El programa verifica si el juego ha terminado. Existen cuatro posibles veredictos: el juego continúa, el juego termina en empate, tu ganas, o la máquina gana.
- La máquina responde con su movimiento y se verifica el estado del juego.
- No se debe implementar algún tipo de inteligencia artificial, la máquina elegirá un cuadro de manera aleatoria, eso es suficiente para este juego.

El ejemplo del programa es el siguiente:

88<row, col = random.choice(free)

89<board[row][col] = 'X'

90<

91<# Inicialización del juego

92<board = [

93< [1, 2, 3],

94< [4, 'X', 6], # La máquina comienza colocando 'X' en el centro

95< [7, 8, 9]

96<]

97<

98<human_turn = True # Turno inicial del humano (después de la máquina)

99<

100<while True:

101< DisplayBoard(board)

102< if human_turn:

103< EnterMove(board)

104< winner = VictoryFor(board, 'O')

105< if winner != None:

106< DisplayBoard(board)

107< print("¡Has Ganado!")

108< break

109< human_turn = False

110< else:

111< DrawMove(board)

112< winner = VictoryFor(board, 'X')

113< if winner != None:

Console >...

| | | | |

| | | | |

| O | 2 | | X |

| | | | |

| | | | |

| 4 | X | 6 |

| | | | |

| | | | |

| X | O | 9 |

| | | | |

|La máquina ha ganado!

Ricardo Daniel

1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome